SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12 COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

78%

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I Abinavah 192521496 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature [illegible] unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	5	4	4	2	18
Q2	4	5	3	2	1.5	15.5
Q3	4	3	2	3	2	14
Q4	3	4	4	4	1.5	16.5
Q5	2	5	3	2	2	14

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	4 Clearly understands the problem and accurately identifies all requirements	3 Good understanding with minor gaps	2 Basic understanding; some requirements unclear	1 Poor understanding or misinterpretation of the problem
Application of Concepts	30	6 Accurately applies appropriate concepts to solve complex problems	5 Applies relevant concepts correctly with minor errors	4 Applies basic concepts with limited accuracy	1-2 Fails to apply appropriate concepts
Problem-Solving Approach	20	4 Logical, wellstructured, and efficient approach	3 Mostly logical approach with minor gaps	2 Basic approach with some inconsistencies	1 Illogical or unclear approach
Accuracy of Solution	20	4 Completely correct solution with proper justification	3 Mostly correct with minor errors	2 Partially correct solution	1 Incorrect or incomplete solution

18

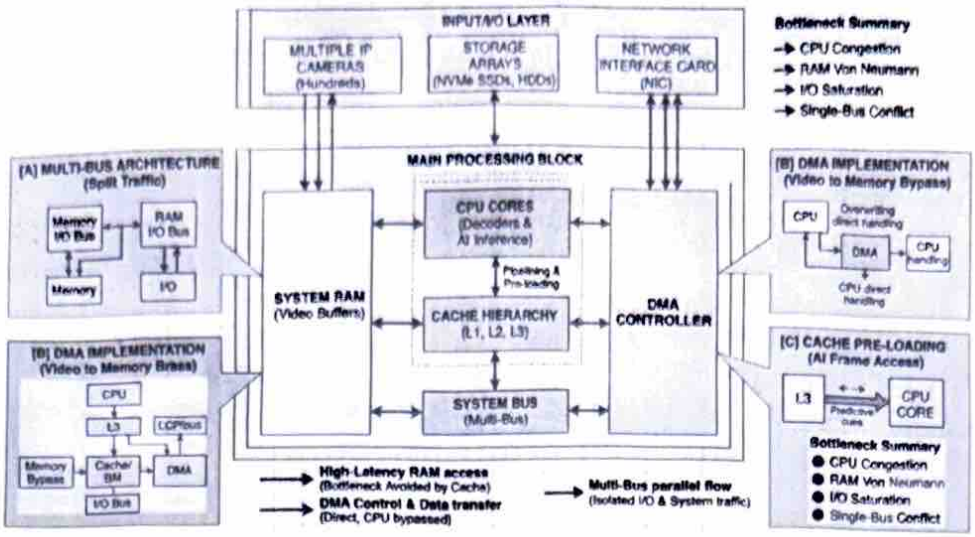
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

ANSWERS:

1. In a smart city surveillance system processing 24/7 high-definition video feeds, handling massive real-time data streams creates severe hardware bottlenecks. When hundreds of cameras stream concurrently, the system's central processing unit, memory modules, and internal buses experience data traffic congestion. Left unoptimized, these bottlenecks result in dropped frames, high latency, and delayed threat detection, compromising critical public safety responses.

Core System Components and Bottleneck Mechanisms

- **Central Processing Unit (CPU)**
  - **Function:** Decodes incoming H.264/H.265 compressed video frames and runs object detection AI algorithms.
  - **Bottleneck:** Complex deep learning inference models require extensive compute cycles per frame. When the CPU is forced to handle basic I/O data routing alongside frame processing, execution time increases significantly.
- **Main Memory (RAM)**
  - **Function:** Serves as the temporary buffer storing active frame buffers for live camera feeds.
  - **Bottleneck:** High latency or insufficient bus bandwidth to RAM causes **Von Neumann bottlenecks**. The CPU stalls and sits idle waiting for frame data to load from memory into internal registers.
- **Input/Output (I/O) Devices**
  - **Function:** Network Interface Cards (NICs) continuously receive incoming video packets from IP cameras while storage arrays (NVMe/SATA SSDs) archive continuous footage.
  - **Bottleneck:** Concurrent read/write requests saturate I/O channels, causing buffer overflows and dropped packets.
- **Bus Architectures: Single vs. Multi-Bus**
  - **Single-Bus Architecture:** CPU, RAM, and I/O controllers share a single system bus. Massive video streams from network ports compete directly with CPU-memory read/write cycles, causing severe system-wide traffic congestion.
  - **Multi-Bus Architecture:** Isolates system traffic using dedicated channels (e.g., a high-speed system bus for CPU-Cache/RAM communication and a separate expansion/PCIe I/O bus for storage and camera data feeds).



Hardware Optimization Strategies for Lag-Free Surveillance

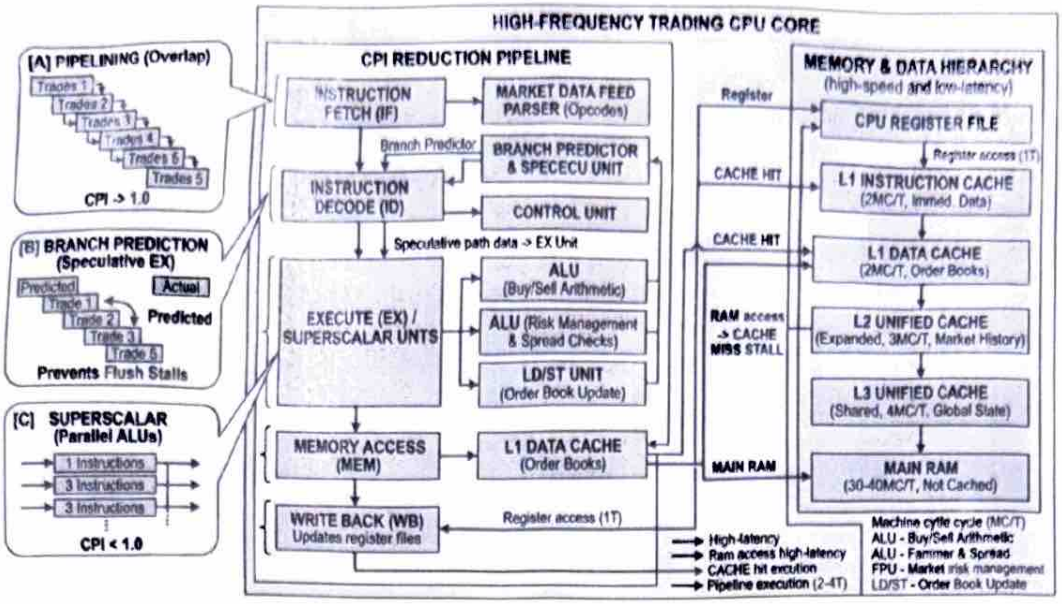
- **Transition to Multi-Bus Architecture:** Separates high-frequency CPU-to-memory traffic from heavy camera streaming I/O. This allows data transfers to occur simultaneously on distinct channels without bus contention.
- **Implement Direct Memory Access (DMA):** Enables Network Interface Cards and storage drives to transfer incoming video frames directly to main memory without routing through the CPU. The CPU remains completely free to run AI threat detection algorithms.
- **Utilize Pipelining and Cache Hierarchies:** Pre-loads sequential video frames into high-speed L1/L2/L3 cache buffers using instruction and data pipelining, ensuring the processing cores never stall waiting on main memory access.

Eliminating processing delays in large-scale smart city surveillance requires relieving hardware bottlenecks across the data path. By upgrading from a single-bus to a multi-bus architecture, offloading frame transfers to Direct Memory Access (DMA) controllers, and leveraging advanced cache hierarchies, the processing unit maintains real-time AI analytics without packet loss or video lag.

2. In high-frequency trading (HFT) platforms processing millions of transactions per second, minimizing Cycles Per Instruction (CPI) is critical. CPI measures the average number of clock cycles required by a CPU core to execute an instruction. High CPI creates processing latency that causes trade execution stalls, slip prices, and lost arbitrage opportunities in microsecond-level order matching.

CPI Bottlenecks in Financial Transaction Pipelines

- **Fetch-Decode-Execute Cycle Overhead**
  - **Fetch:** The CPU retrieves market data parser opcodes and order execution instructions from system memory. Memory bottlenecks during high tick-volume spikes stall fetching.
  - **Decode:** The Control Unit decodes financial operations (e.g., verifying risk limits, checking bid-ask spreads, or parsing market data protocols).
  - **Execute:** The Arithmetic Logic Unit (ALU) computes ledger balances, updates order book structures, and writes back execution acknowledgments.
- **Root Causes of High CPI**
  - **Branch Mispredictions:** Frequent conditional logic (e.g., evaluating if stock price exceeds limit boundaries) causes pipeline flushes when speculative paths fail.
  - **Cache Misses:** Fetching market data from high-latency main RAM instead of high-speed cache causes CPU memory stall cycles.
  - **Data Dependencies:** Sequential calculations (where step N depends on step $N-1$ output) create pipeline hazard stalls.



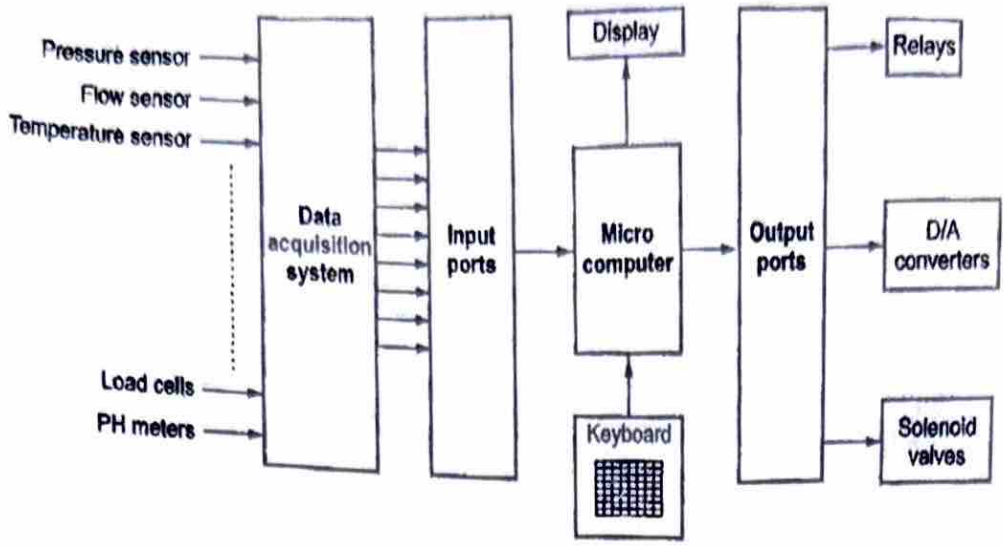
Microarchitectural Strategies to Reduce CPI

- **Instruction Pipelining**
  - Divides instruction execution into distinct, overlapped stages (Fetch, Decode, Execute, Writeback).
  - *Impact:* Allows the CPU to fetch a new incoming order instruction while decoding the current order and writing back the previous order, driving effective CPI closer to 1.0.
- **Advanced Branch Prediction**
  - Uses speculative execution hardware to predict the outcome of order limit checks before conditional execution finishes.
  - *Impact:* Prevents pipeline flushes and pipeline stall delays on repetitive market data processing loops.
- **Expanded L1/L2/L3 Cache Hierarchy**
  - Keeps active order book data structures, client accounts, and risk rules cached in high-speed, low-latency SRAM directly adjacent to CPU cores.
  - *Impact:* Dramatically reduces cache misses, eliminating hundreds of idle CPU wait cycles per trade.
- **Superscalar Execution Units**
  - Deploys multiple parallel execution units (ALUs and FPUs) within a single CPU core.
  - *Impact:* Executes multiple non-dependent trade verification instructions simultaneously within a single clock cycle, bringing the effective CPI below 1.0 (Instructions Per Cycle > 1).

Conclusion

Minimizing CPI within transaction processing pipelines bridges the gap between hardware hardware execution limits and algorithmic market demands. By combining instruction pipelining, speculative branch prediction, and multi-tiered high-speed cache management, high-frequency trading platforms eliminate CPU wait cycles to deliver deterministic sub-microsecond transaction latency.

3. In an 8085-based industrial temperature controller, the microprocessor acts as the central control unit. It continuously fetches temperature data from sensors via Analog-to-Digital Converters (ADCs), evaluates these readings against preconfigured threshold limits, and outputs control signals to devices like cooling fans, heating elements, or alarm relays. Choosing the appropriate addressing mode for each task directly dictates code execution speed, program size, and memory efficiency, ensuring the control loop reacts in real time without failure.



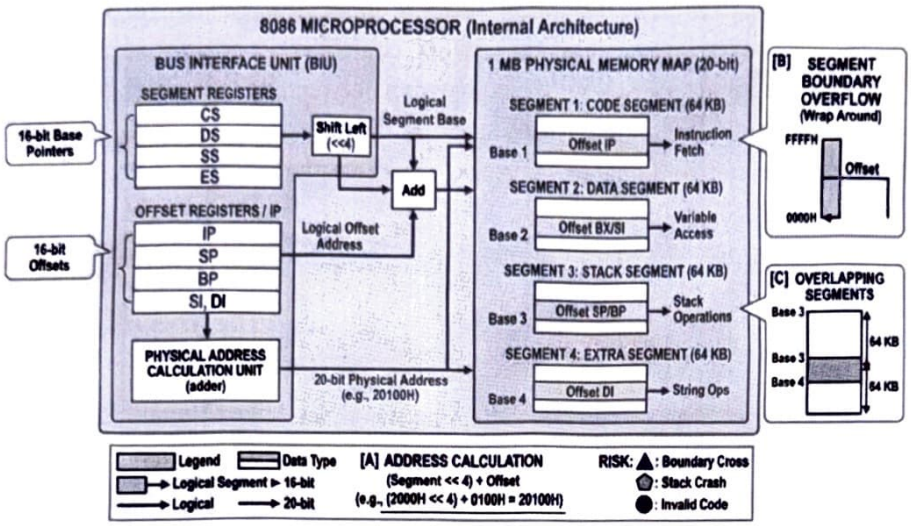
4. In an 8086-based computing architecture, the processor addresses a total memory space of 1 MB (2^20 bytes) using a segmented memory model. To overcome the limitation of 16-bit internal registers, the 1 MB memory space is divided into logical, non-overlapping or overlapping 64 KB segments. Managing these segments properly ensures that program instructions, data variables, and call stack frames remain safely isolated. Poor segment management leads to corrupted values, stack overflows, and severe runtime system crashes.

Core Memory Segments in 8086 Architecture

- **Code Segment (CS)**
    - **Function:** Stores the machine code instructions currently being fetched and executed by the CPU.
    - **Addressing:** Combined with the Instruction Pointer (IP) to generate the physical address (Physical Address = CS * 16 + IP).
    - **Risk:** Writing data into the Code Segment or miscalculating offsets can overwrite program code, causing invalid opcode execution.
- **Data Segment (DS)**
    - **Function:** Holds global variables, dynamic program data, and static constants.
    - **Addressing:** Accessed using general-purpose registers (BX, SI, DI) or direct memory offsets.
    - **Risk:** Array boundary overflows in the Data Segment can bleed into adjacent memory, corrupting neighbouring variables.
- ***Stack Segment* (SS)**
    - ***Function:*** *Allocates a dedicated high-speed memory area for temporary variable storage, sub-routine parameters, and procedure return addresses.*
    - ***Addressing:*** *Managed using the Stack Pointer (SP) and Base Pointer (BP).*

- **Risk:** Stack underflows or overflows alter stored return addresses, causing execution to jump to arbitrary memory locations upon returning from a procedure.

- **Extra Segment (ES)**
  - **Function:** Functions as a secondary data segment specifically optimized for string manipulation and large memory buffer operations.
  - **Addressing:** Paired with the Destination Index (DI) register during string instructions (MOVSB, STOSB).
  - **Risk:** Uninitialized ES segment pointers during block memory transfers write output data into active code or data regions.



Physical Memory Calculation in 8086

The 8086 generates a 20-bit Physical Address from two 16-bit values (Segment Base Address and Offset Address) using the internal Bus Interface Unit (BIU):

Physical Address = (Segment Register * 10H) + Offset Register
Physical Address = (Segment Register << 4) + Offset Register

Example: If CS = 2000H and IP = 0100H:

1. Shift CS Left by 4 bits: 2000H * 10H = 20000H
2. Add Offset (IP): 20000H + 0100H = **20100H** (20-bit Physical Address in RAM)

Common Implementation Risks & Mitigation Strategies

- **Segment Boundary Overflows:** An offset exceeding FFFFH (64 KB) wraps around to 0000H within the same segment, overwriting critical data at the start of that segment instead of accessing higher memory.

- **Stack Pointer Misalignment:** Unbalanced PUSH and POP operations shift the Stack Pointer (SP). Returning from a function (RET) pulls corrupt pointer data into IP, causing the system to crash.

- **Overlapping Segments:** Assigning base addresses that place segments less than 64 KB apart leads to overlap, where stack updates silently overwrite data segment variables.

Conclusion

Managing the 64 KB logical segments in the 8086 microprocessor guarantees deterministic execution and robust memory protection. By properly initializing segment registers (CS, DS, SS, ES), enforcing boundary checks, and tracking the stack pointer, system designers maintain memory isolation and prevent catastrophic runtime crashes.

5. In modern data communication systems, packet headers, payloads, and control bytes are frequently represented in hexadecimal format to simplify debugging, human readability, and transmission logging. However, microprocessors, network controllers, and execution pipelines process data purely in binary logic gates. Accurate conversion between hexadecimal and binary is critical; even a single-bit misinterpretation distorts opcodes, memory addresses, or control flags, corrupting instruction execution and destabilizing real-time communications.

Importance of Hexadecimal-to-Binary Conversion in Data Communications

- **Hex-to-Binary Direct Mapping**
    - Hexadecimal provides a 1-to-4 mapping where each hex digit directly corresponds to a 4-bit binary nibble (e.g., 0xA = 1010_2, 0xF = 1111_2).
    - This compact representation reduces lengthy 32-bit or 64-bit binary packets into concise, human-readable strings without introducing rounding or non-integer conversion overhead.
- **Header and Payload Protocol Parsing**
    - Network protocols rely on bit-level fields within packet headers (such as IP addresses, TCP flags, port numbers, and frame check sequences).
    - Engineers use hex logs to inspect raw packet bytes; translating these bytes back to binary ensures that underlying hardware registers receive exact bit patterns for packet filtering and routing logic.
- **Memory-Mapped I/O Alignment**
    - Network Interface Cards (NICs) use memory-mapped I/O (MMIO) addresses expressed in hexadecimal (e.g., 0x8000_0000).
    - Proper translation guarantees that the CPU targets correct physical address spaces in system RAM during Direct Memory Access (DMA) video or packet transfers.

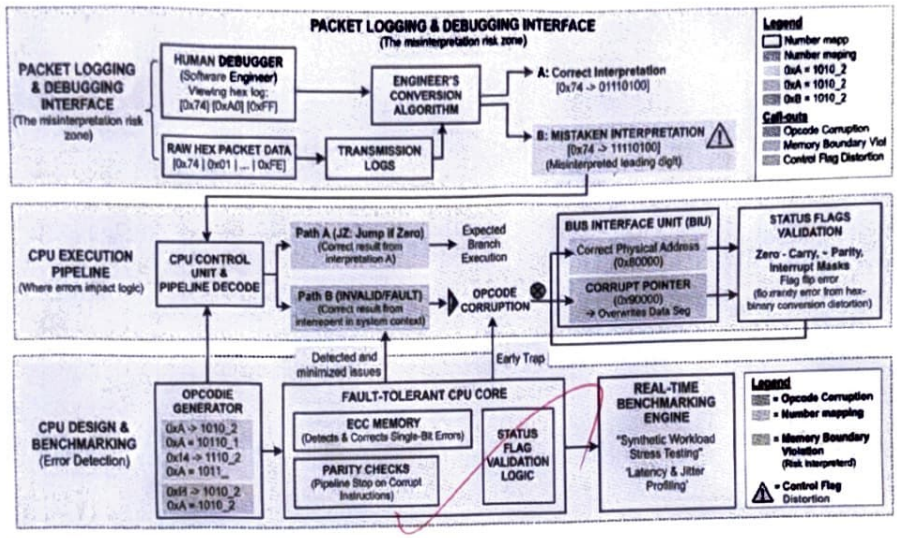
Impact of Number System Conversion Errors on Instruction Execution

- **Opcode Corruption & Invalid Instruction Traps**
    - If a packet contains execution opcodes, misinterpreting a single hex digit alters the instruction bit pattern. For instance, misinterpreting 0x74 (JZ - Jump if Zero) as 0x75 (JNZ

- Jump if Not Zero) flips conditional jump logic, triggering improper branch execution or invalid opcode hardware interrupts.

- **Memory Boundary Violation & Pointer Corruption**
  - An error in converting hex address offsets to binary causes the CPU's Bus Interface Unit (BIU) to generate incorrect 20-bit or 32-bit physical addresses.
  - This results in out-of-bounds reads/writes, overwriting critical data segment variables or triggering segmentation faults and system crashes.

- **Control Flag and Mask Bit Distortions**
  - Bitwise masking operations (e.g., AND, OR, XOR) rely on precise binary masks derived from hex configurations.
  - A conversion error in a bit mask (e.g., using 0xFE / 1111_1110_2 instead of 0xFF / 1111_1111_2) inadvertently clears critical interrupt mask or status flags, disabling real-time packet arrival notifications.



Role of CPU Design and Benchmarking in Detecting and Minimizing Errors

- **Hardware Parity and Error Correcting Code (ECC)**
  - Modern CPU buses and memory hierarchies incorporate parity bits and ECC logic.
  - If a conversion bug or transmission error flips bits in data buses or registers, hardware-level ECC automatically detects and corrects single-bit errors before execution phases begin.

- **Parity and Zero Flag Pipeline Checks**
  - CPU Control Units leverage internal status registers (such as the Parity Flag, Zero Flag, or Carry Flag) immediately after ALU decoding.
  - Validation logic checks incoming packet bytes for structural integrity before passing parsed instructions to execution units, trapping corrupt frame structures early.

- **Benchmarking and Real-Time Diagnostic Testing**

- **Synthetic Workload Stress Testing:** Benchmarking tools run simulated high-throughput packet streams through network execution pipelines to monitor Instructions Per Cycle (IPC) degradation, branch mispredictions, and cache miss rates caused by corrupted control logic.
- **Real-Time Latency & Jitter Profiling:** Benchmarking identifies processing delays induced by unexpected exception-handling traps, ensuring hardware and software layers maintain sub-millisecond execution times in real-time communications.

Conclusion

Hexadecimal-to-binary conversion is foundational to data communication and processor execution pipelines. Misinterpreting hex values distorts underlying binary bit patterns, leading to opcode corruption, misdirected memory access, and execution halts. Combining fault-tolerant CPU design—such as ECC memory, automated status flag validation, and robust pipeline decoding—with rigorous real-time benchmarking guarantees reliable, low-latency packet processing across industrial and network systems.